# Sprint 10: Accessibility Foundations and Assistive Technology

## Full Structured Note in Simple Words

---

## 1. Where You Are Now

You have already completed **Sprint 1 to Sprint 9**.

That means you already know the main HTML foundation:

| Sprint | Main topic you learned |
|---|---|
| Sprint 1 | HTML elements, tags, nesting, attributes, void elements |
| Sprint 2 | Full HTML boilerplate, `head`, metadata |
| Sprint 3 | Text meaning with elements like `em`, `strong`, `abbr`, `time`, `code` |
| Sprint 4 | Links, paths, `mailto:`, `tel:`, navigation |
| Sprint 5 | Images, audio, video, SVG, iframe, media responsibility |
| Sprint 6 | Semantic layout: `header`, `nav`, `main`, `section`, `article`, `footer` |
| Sprint 7 | Form basics, labels, inputs, placeholders |
| Sprint 8 | Buttons, validation, disabled, readonly, focus states |
| Sprint 9 | Tables, captions, table structure, debugging, validators |

Now Sprint 10 moves you from this question:

> Can I write valid HTML?

To this better question:

> Can every type of user understand and use my page?

That is the main purpose of accessibility.

---

## 2. Sprint 10 Main Goal

Sprint 10 is about:

> Understanding who accessibility helps and why semantic HTML is the first layer of accessibility.

Accessibility means making websites usable for as many people as possible.

It is not only for blind users. It supports many users, including people with:

- Visual disabilities
- Auditory disabilities
- Motor disabilities
- Cognitive disabilities

It also helps people in temporary or difficult situations.

Examples:

- A person using a phone outside in bright sunlight benefits from good contrast.
- A person with a broken mouse benefits from keyboard navigation.
- A person watching a video in a noisy place benefits from captions.
- A person who is tired or stressed benefits from simple instructions.

So, accessibility means:

> Everyone should be able to receive information, use the page, understand the page, and access it with different tools.

---

## 3. What Accessibility Means in Web Development

In web development, accessibility means building web pages that can be used by different people with different needs.

A website should not depend only on:

- Perfect eyesight
- Perfect hearing
- A mouse
- Fast reading ability
- Complex memory
- One type of device

A good website should work for many users and many situations.

Accessibility starts from HTML because HTML gives the page structure and meaning.

Good HTML helps:

- Browsers
- Screen readers
- Search engines
- Keyboard users
- Voice control software
- Magnification tools
- Other assistive technologies

---

## 4. Main Disability Categories

Sprint 10 discusses four major categories of disabilities that can affect web use.

---

## 4.1 Visual Difficulties

Visual difficulties affect how users see or read web content.

Examples include:

- Blindness
- Low vision
- Color blindness
- Difficulty reading small text

These users may use:

- Screen readers
- Screen magnifiers
- Braille output
- High contrast settings
- Large text settings

### HTML Concern for Visual Users

Images should have useful `alt` text.

Bad example:

```
<img src="product.jpg">
```

Problem:

The image has no text alternative. A screen reader user may not know what the image means.

Better example:

```
<img src="product.jpg" alt="Black leather office chair with arm rests.">
```

Why this is better:

The `alt` text explains the important image information.

### Simple Rule

> If important information is inside an image, provide text that explains it.

---

## 4.2 Auditory Difficulties

Auditory difficulties affect how users hear audio or video content.

Examples include:

- Deafness
- Hard of hearing
- Difficulty understanding spoken audio

These users may need:

- Captions
- Subtitles
- Transcripts
- Visual instructions

### HTML Concern for Auditory Users

Videos with speech should have captions.

Example:

```
<video controls src="lesson.mp4">
  <track src="captions.vtt" kind="captions" srclang="en" label="English">
</video>
```

Why this helps:

A user who cannot hear the video can read the captions.

### Simple Rule

> If audio gives important information, provide a text alternative.

---

## 4.3 Motor Difficulties

Motor difficulties affect how users move, click, tap, or type.

Some users may find it hard to use:

- A mouse
- A touchscreen
- Small buttons
- Drag-and-drop actions
- Complex gestures

They may use:

- Keyboard only
- Trackball
- Joystick
- Touchpad
- Voice control
- Special input devices

### HTML Concern for Motor Users

Use real HTML controls.

Bad example:

```
<div onclick="submitForm()">Submit</div>
```

Problem:

A `div` is not naturally a button. It may not work well with keyboard or assistive technology.

Better example:

```
<button type="submit">Submit</button>
```

Why this is better:

A real `button` already supports keyboard interaction and has correct meaning.

### Simple Rule

> Do not create mouse-only interfaces.

---

## 4.4 Cognitive Difficulties

Cognitive difficulties affect how users understand, remember, focus, or process information.

Examples include difficulty with:

- Memory
- Attention
- Reading
- Understanding complex words
- Following unclear instructions
- Processing too much information at once

### HTML and Content Concern for Cognitive Users

Use clear headings and simple instructions.

Example:

```
<h1>Job Application Form</h1>
<p>Please complete the form below. Required fields are marked clearly.</p>
```

Why this helps:

The heading tells the user what the page is about. The paragraph explains what to do.

### Simple Rule

> Make the page easy to understand, not just technically correct.

---

## 5. WCAG

WCAG means:

> Web Content Accessibility Guidelines

WCAG gives guidelines for making web content accessible.

You can think of WCAG as a standard that explains how to make websites better for different users.

WCAG is built around four main principles called **POUR**.

---

## 6. POUR Principles

POUR stands for:

| Letter | Meaning | Simple explanation |
|---|---|---|
| P | Perceivable | Users must be able to receive the information |
| O | Operable | Users must be able to use the interface |
| U | Understandable | Users must understand the content and controls |
| R | Robust | Browsers and assistive tools must understand the code |

These four principles help you think about accessibility clearly.

---

## 6.1 P — Perceivable

Perceivable means:

> Users must be able to get the information.

If the user cannot see or hear something, you should provide another way to understand it.

### Example Problem

An image contains important sale information, but the image has no `alt` text.

Bad example:

```
<img src="sale-banner.png">
```

Problem:

A screen reader user cannot understand the sale message.

Better example:

```
<img src="sale-banner.png" alt="Summer sale starts on June 1 with up to 40%
off.">
```

Why this is better:

The important message is now available as text.

### More Perceivable Examples

Good perceivable choices include:

- Adding `alt` text to meaningful images
- Adding captions to videos
- Providing transcripts for audio
- Using real text instead of images of text
- Making content readable when zoomed

### Simple Memory Rule

> If the user cannot see it or hear it, give another way to receive it.

---

## 6.2 O — Operable

Operable means:

> Users must be able to use the page.

A page should not require only a mouse.

Users should be able to move through the page and activate controls using different tools.

### Example Problem

A page uses a clickable `div` for navigation.

Bad example:

```
<div onclick="goHome()">Home</div>
```

Problem:

This is mouse-based. It may not behave like a real link for keyboard users.

Better example:

```
<a href="index.html">Home</a>
```

Why this is better:

A real link has built-in navigation meaning.

### More Operable Examples

Good operable choices include:

- Use real links for navigation
- Use real buttons for actions
- Make form fields reachable by keyboard
- Avoid keyboard traps
- Make clickable areas large enough
- Keep navigation easy to use

### Simple Memory Rule

> Users must be able to reach it, focus it, and activate it.

---

## 6.3 U — Understandable

Understandable means:

> Users must understand the content and how to use the page.

Do not make the page unnecessarily confusing.

### Example Problem

The instruction uses complex wording.

Bad example:

```
<p>Proceed with the aforementioned operation.</p>
```

Problem:

This sentence is harder than necessary.

Better example:

```
<p>Click Submit to send your form.</p>
```

Why this is better:

It is direct and easy to understand.

### More Understandable Examples

Good understandable choices include:

- Clear page titles
- Clear headings
- Simple paragraphs
- Clear form labels
- Helpful instructions
- Avoiding vague link text
- Consistent navigation

### Simple Memory Rule

> The user should not have to guess what to do.

---

## 6.4 R — Robust

Robust means:

> Browsers and assistive technologies must be able to understand your HTML.

This is where valid and semantic HTML becomes very important.

### Example Problem

Heading levels are skipped.

Bad example:

```
<h1>Page Title</h1>
<h3>About Us</h3>
```

Problem:

The page jumps from `h1` to `h3`. This can confuse the page outline.

Better example:

```
<h1>Page Title</h1>
<h2>About Us</h2>
```

Why this is better:

The heading structure is logical.

### More Robust Examples

Good robust choices include:

- Use semantic HTML elements
- Use correct heading order
- Use valid HTML structure
- Associate labels with form inputs
- Use real buttons and links
- Avoid broken or confusing markup

### Simple Memory Rule

> Good structure helps assistive tools understand your page.

---

## 7. Semantic HTML and Accessibility

Semantic HTML means using HTML elements according to their meaning.

Examples:

| Use this | Meaning |
| --- | --- |
| `nav` | Navigation area |

| Use this | Meaning |
|---|---|
| `main` | Main page content |
| `header` | Introductory content or top area |
| `footer` | Footer content |
| `section` | Related group of content |
| `article` | Self-contained content |
| `button` | Action button |
| `a` | Link to another place |

Semantic HTML helps accessibility because assistive technologies can understand the structure.

Bad example:

```
<div>
  <div>Home</div>
  <div>About</div>
</div>
```

Better example:

```
<nav>
  <a href="index.html">Home</a>
  <a href="about.html">About</a>
</nav>
```

Why this is better:

The `nav` element tells browsers and assistive technologies that this is a navigation area.

---

## 8. Assistive Technology

Assistive technology means:

> Tools that help people use computers, websites, and apps.

Assistive technology can support users who have visual, auditory, motor, or cognitive difficulties.

Sprint 10 discusses different kinds of assistive technology.

## 8.1 Screen Readers

A screen reader is software that reads the page content aloud or sends it to a braille device.

Screen readers are used by:

- Blind users
- Low-vision users
- Dyslexic users
- Some cognitively disabled users

A screen reader can help users move through:

- Headings
- Links
- Forms
- Tables
- Landmarks like `nav`, `main`, and `footer`

### Examples of Screen Readers

Examples include:

- VoiceOver
- Narrator
- NVDA
- JAWS
- Orca
- Speakup
- TalkBack

### Why Screen Readers Need Good HTML

Screen readers depend on structure.

Good structure:

```
<main>
  <h1>Music Store Page</h1>

  <section>
    <h2>Featured Albums</h2>
    <p>Check out our featured albums below.</p>
```

```
  </section>
</main>
```

This is helpful because:

- `main` shows the main content area.
- `h1` gives the main page topic.
- `h2` gives a section topic.
- `p` gives readable text.

Bad structure:

```
<div>
  <div>Music Store Page</div>
  <div>Featured Albums</div>
</div>
```

This is weaker because the elements do not clearly explain their meaning.

---

## 8.2 Keyboard Users

Some users do not use a mouse.

They may use only a keyboard.

Common keyboard keys include:

- `Tab`
- `Shift + Tab`
- `Enter`
- `Space`
- Arrow keys

Real HTML elements usually support keyboard use naturally.

Good example:

```
<a href="courses.html">View Courses</a>
<button type="submit">Submit</button>
```

These are good because:

- A real link can be focused and opened.

- A real button can be focused and activated.

Weak example:

```
<div>View Courses</div>
<span>Submit</span>
```

These are weak because they do not naturally behave like links or buttons.

### Simple Rule

> Use real HTML controls before making fake controls.

---

## 8.3 Large Print and Braille Keyboards

Some users may use special keyboards.

Examples:

- Large print keyboards
- Braille keyboards

These tools help users enter information and control the computer.

HTML concern:

The page should use normal form controls and clear labels.

Example:

```
<label for="email">Email Address</label>
<input type="email" id="email" name="email">
```

Why this helps:

The label clearly explains the input field.

---

## 8.4 Alternative Pointing Devices

Some users may not use a normal mouse.

They may use:

- Trackballs
- Joysticks
- Touchpads
- Special pointing devices

HTML and design concern:

Controls should be easy to reach and understand.

Good example:

```
<button type="button">Open Course Details</button>
```

Bad example:

```
<span onclick="openDetails()">+</span>
```

Problem:

A tiny plus sign may be hard to target and understand.

---

## 8.5 Screen Magnifiers

A screen magnifier enlarges the page so low-vision users can read it.

When a user zooms in, they may only see a small part of the page at a time.

Problems can happen when:

- Navigation is too large
- Sticky headers block content
- Text does not reflow properly
- Important content is hidden
- Buttons or links become hard to find

### Simple Design Concern

A huge sticky navigation bar can block content when magnified.

Better approach:

- Keep navigation simple
- Avoid covering content
- Use real text
- Avoid fixed layouts that break when zoomed

### Simple Rule

> Your page should still work when the user zooms in.

---

## 8.6 Voice Recognition Software

Voice recognition software lets users control the computer using speech.

A user may say commands like:

> Click Submit

or

> Click Contact Us

So button text and link text should be clear.

Bad example:

```
<button>Go</button>
```

Problem:

The word “Go” is vague. The user may not know where it goes.

Better example:

```
<button>Submit Application</button>
```

Why this is better:

The button text clearly describes the action.

Bad link example:

```
<a href="details.html">Click here</a>
```

Better link example:

```
<a href="details.html">Read course details</a>
```

### Simple Rule

> Link and button text should clearly explain the action.

---

## 9. Accessibility Audit Tools

Accessibility audit tools help find some accessibility problems.

Examples include:

- Lighthouse
- WAVE
- IBM Equal Accessibility Checker

These tools can check things like:

- Missing `alt` attributes
- Low contrast issues
- Missing form labels
- Heading structure problems
- Some ARIA problems
- Some keyboard issues

But automated tools are limited.

They cannot fully decide whether the page is truly usable.

---

## 10. Why Automated Tools Are Not Enough

Automated tools can find some problems, but not all problems.

For example, a tool may check whether an image has `alt` text.

This may pass:

```
<img src="chart.png" alt="image">
```

But the `alt` text is poor because it does not explain the chart.

Better:

```
<img src="chart.png" alt="Sales increased from January to March.">
```

A tool may not fully understand whether:

- The `alt` text is meaningful
- The link text makes sense in context
- The page is easy to understand
- Keyboard navigation feels natural
- A screen reader user can complete the task easily
- The content is too confusing

So accessibility testing should include:

- Automated tools
- Manual keyboard testing
- Screen reader testing
- User testing where possible

### Simple Rule

> Automated tools help, but humans must still test the experience.

---

## 11. Accessibility Starts Before ARIA

ARIA is not the main topic of Sprint 10. ARIA comes later.

For Sprint 10, remember this important idea:

> Start with semantic HTML before using advanced accessibility attributes.

Good HTML first:

```
<button type="button">Search</button>
```

Avoid fake controls:

```
<div role="button">Search</div>
```

Why?

A real `button` already has correct meaning and behavior.

ARIA should not be used to fix bad HTML when a native HTML element already exists.

---

## 12. Accessibility-First HTML Patterns

Below are simple HTML patterns that support accessibility.

---

## 12.1 Semantic Page Structure

```
<header>
  <h1>Accessibility Foundations</h1>

  <nav>
    <a href="index.html">Home</a>
    <a href="accessibility.html">Accessibility</a>
    <a href="contact.html">Contact</a>
  </nav>
</header>

<main>
  <section>
    <h2>Perceivable</h2>
    <p>Users must be able to get the information.</p>
  </section>
</main>

<footer>
  <p>&copy; 2026 HTML Learning Page</p>
</footer>
```

Why this is good:

- `header` gives top page content.
- `nav` gives navigation meaning.
- `main` gives main content.
- `section` groups related content.
- `footer` gives footer content.

## 12.2 Clear Heading Structure

Good:

```
<h1>Accessibility Foundations</h1>
<h2>Perceivable</h2>
<h2>Operable</h2>
<h2>Understandable</h2>
<h2>Robust</h2>
```

Bad:

```
<h1>Accessibility Foundations</h1>
<h3>Perceivable</h3>
<h5>Operable</h5>
```

Why the bad version is wrong:

It skips heading levels and creates a confusing outline.

---

## 12.3 Image With Useful Alt Text

```
<img src="course-cover.jpg" alt="Student learning HTML on a laptop.">
```

Why this is good:

The image meaning is available as text.

---

## 12.4 Real Link for Navigation

```
<a href="index.html">Home</a>
```

Why this is good:

The browser knows it is a link.

---

## 12.5 Real Button for Action

```
<button type="button">Open Course Details</button>
```

Why this is good:

The browser knows it is a button.

---

## 12.6 Clear Form Label

```
<label for="name">Your Name</label>
<input type="text" id="name" name="name">
```

Why this is good:

The label is connected to the input.

---

## 12.7 Video With Captions

```
<video controls src="lesson.mp4">
  <track src="captions.vtt" kind="captions" srclang="en" label="English">
</video>
```

Why this is good:

Users who cannot hear the audio can read captions.

---

# 13. Common Beginner Mistakes in Sprint 10

---

## Mistake 1: Thinking Accessibility Is Only for Blind Users

Wrong idea:

> Accessibility only means screen readers.

Correct idea:

Accessibility supports visual, auditory, motor, and cognitive needs.

---

## Mistake 2: Using `div` for Everything

Bad:

```
<div>
  <div>Home</div>
  <div>About</div>
</div>
```

Better:

```
<nav>
  <a href="index.html">Home</a>
  <a href="about.html">About</a>
</nav>
```

Reason:

`nav` and `a` give real meaning.

---

## Mistake 3: Creating Mouse-Only Actions

Bad:

```
<div onclick="goHome()">Home</div>
```

Better:

```
<a href="index.html">Home</a>
```

Reason:

A real link is more accessible than a clickable `div`.

---

## Mistake 4: Using Images of Text

Bad:

```
<img src="welcome-text.png" alt="">
```

Better:

```
<h1>Welcome to Our Website</h1>
```

Reason:

Real HTML text is easier for:

- Screen readers
- Translation tools
- Zoom
- Search engines
- Copying text

---

## Mistake 5: Trusting Only Automated Tools

Wrong idea:

> Lighthouse score is 100, so the page is fully accessible.

Correct idea:

> Automated tools help, but manual testing is still needed.

---

## Mistake 6: Skipping Heading Levels

Bad:

```
<h1>Page</h1>
<h3>Products</h3>
```

Better:

```
<h1>Page</h1>
<h2>Products</h2>
```

Reason:

Screen readers use headings to understand page structure.

---

## Mistake 7: Vague Link or Button Text

Bad:

```
<a href="details.html">Click here</a>
<button>Go</button>
```

Better:

```
<a href="details.html">Read course details</a>
<button>Submit Application</button>
```

Reason:

The user can understand the action without guessing.

---

# 14. Sprint 10 Debug Practice

### Broken Code

```
<div onclick="goHome()">Home</div>

<img src="sale-text.png">

<h1>Page</h1>
<h3>Section with skipped level</h3>

<p>Click the tiny icon to continue.</p>
```

---

## 14.1 What Is Wrong?

| Problem | Why it is bad |
|---|---|
| `<div onclick="goHome()">Home</div>` | It creates a mouse-based navigation action instead of a real link |
| `<img src="sale-text.png">` | The image has no `alt` text |
| `h1` then `h3` | The heading level is skipped |
| “Click the tiny icon” | The instruction is vague and may be difficult for some users |

---

## 14.2 Fixed Beginner Version

```
<nav>
  <a href="index.html">Home</a>
</nav>

<img src="sale-text.png" alt="Summer sale starts today with discounts on selected items.">

<h1>Page</h1>
<h2>Section with correct heading level</h2>

<p>Select the Continue button to move to the next step.</p>
<button type="button">Continue</button>
```

---

## 14.3 Why This Fixed Version Is Better

| Original issue | Fix |
|---|---|
| Mouse-only `div` | Used real `a` link |
| Image without `alt` | Added meaningful `alt` text |
| Skipped heading level | Changed `h3` to `h2` |
| Vague instruction | Added clear instruction and real button |

---

## 15. Mini Code Example for Sprint 10

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Accessibility Foundations</title>
</head>
<body>
  <header>
    <h1>Accessibility Foundations</h1>

    <nav>
      <a href="index.html">Home</a>
      <a href="accessibility.html">Accessibility</a>
      <a href="contact.html">Contact</a>
    </nav>
  </header>

  <main>
    <section>
      <h2>Perceivable</h2>
      <p>Users must be able to get the information.</p>
      <img src="course-cover.jpg" alt="Student learning HTML on a laptop.">
    </section>

    <section>
      <h2>Operable</h2>
      <p>Users must be able to use the page with a keyboard or other input
device.</p>
      <button type="button">Open Course Details</button>
    </section>

    <section>
      <h2>Understandable</h2>
      <p>Content should use clear words and simple instructions.</p>
    </section>

    <section>
      <h2>Robust</h2>
      <p>Use valid semantic HTML so browsers and assistive tools can understand
the page.</p>
    </section>
  </main>
```

```
  <footer>
    <p>&copy; 2026 HTML Learning Page</p>
  </footer>
</body>
</html>
```

---

## 16. Explanation of the Mini Code Example

`<!DOCTYPE html>`

This tells the browser to use modern HTML.

`<html lang="en">`

This tells browsers and assistive tools that the page language is English.

`<header>`

This contains the top part of the page.

`<nav>`

This marks the navigation area.

`<main>`

This marks the main page content.

`<section>`

Each section groups related content.

`h1` **and** `h2`

The heading structure is logical:

- `h1` is the page title.
- `h2` is used for main sections.

`alt`

The image has text alternative information.

`<button>`

A real button is used for an action.

`<footer>`

This marks the bottom page area.

---

## 17. Sprint 10 Practice Task

Create a file named:

```
sprint-10-accessibility-foundations.html
```

Build a page with the following:

1. Full HTML boilerplate
2. `h1` called **Accessibility Foundations**
3. Four `h2` sections:
4. Perceivable
5. Operable
6. Understandable
7. Robust
8. One simple paragraph under each `h2`
9. One semantic `nav`
10. One image with meaningful `alt`
11. One real `button`
12. A final section called **Testing Limits**
13. In the Testing Limits section, explain why automated tools are not enough

---

## 18. Practice Task Starter Structure

You can use this as a guide, but try to write it yourself first.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
  <title>Accessibility Foundations</title>
</head>
<body>
  <header>
    <h1>Accessibility Foundations</h1>

    <nav>
      <a href="index.html">Home</a>
      <a href="about.html">About</a>
      <a href="contact.html">Contact</a>
    </nav>
  </header>

  <main>
    <section>
      <h2>Perceivable</h2>
      <p>Users must be able to receive the information on the page.</p>
      <img src="accessibility.jpg"
alt="Person using a laptop with accessibility tools.">
    </section>

    <section>
      <h2>Operable</h2>

<p>Users must be able to use the page with a keyboard, mouse, or assistive
device.</p>
      <button type="button">Learn More</button>
    </section>

    <section>
      <h2>Understandable</h2>
      <p>Content should use clear words and simple instructions.</p>
    </section>

    <section>
      <h2>Robust</h2>
      <p>HTML should be valid and semantic so browsers and assistive
technologies can understand it.</p>
    </section>

    <section>
      <h2>Testing Limits</h2>
      <p>Automated tools can find some problems, but manual testing is also
needed because tools cannot fully judge real user experience.</p>
    </section>
  </main>

  <footer>
```

```
    <p>&copy; 2026 Accessibility Practice Page</p>
  </footer>
</body>
</html>
```

---

## 19. Sprint 10 Checklist

Use this checklist to review your work.

| Question | Yes/No |
|---|---|
| Did I use a complete HTML boilerplate? | |
| Did I use semantic layout elements? | |
| Did I use one clear `h1`? | |
| Did I use correct heading order? | |
| Did I avoid skipping heading levels? | |
| Did I use real links for navigation? | |
| Did I use a real button for action? | |
| Did every meaningful image have useful `alt` text? | |
| Did I avoid mouse-only interaction? | |
| Did I explain POUR clearly? | |
| Did I remember that automated tools are not enough? | |

---

## 20. Sprint 10 Revision Questions

Answer these before moving to Sprint 11.

1. What does web accessibility mean?
2. Why is accessibility not only for blind users?
3. Name four disability categories that affect web use.
4. What does WCAG stand for?
5. What does POUR stand for?
6. What does Perceivable mean?
7. What does Operable mean?
8. What does Understandable mean?
9. What does Robust mean?
10. Give one example of perceivable HTML.

11. Give one example of operable HTML.
12. Why is a real `button` better than a clickable `div`?
13. Why should images have `alt` text?
14. Why should video content have captions?
15. Who uses screen readers besides blind users?
16. Name three screen readers.
17. Why is semantic HTML important for screen readers?
18. Why should heading levels not be skipped?
19. Name three accessibility audit tools.
20. Why are automated accessibility tools not enough?

---

## 21. Key Terms

| Term | Simple meaning |
|---|---|
| Accessibility | Making websites usable by many different people |
| Assistive technology | Tools that help users use websites and computers |
| WCAG | Web Content Accessibility Guidelines |
| POUR | Perceivable, Operable, Understandable, Robust |
| Perceivable | Users can receive the information |
| Operable | Users can use the page controls |
| Understandable | Users can understand the content and actions |
| Robust | Browsers and assistive tools can understand the HTML |
| Screen reader | Software that reads page content aloud or outputs braille |
| Screen magnifier | Software that enlarges the screen |
| Voice recognition | Software that allows users to control the computer by speaking |
| Semantic HTML | HTML that uses elements according to their meaning |
| `alt` text | Text alternative for an image |
| Keyboard navigation | Moving through a page using keyboard keys |
| Audit tool | Tool that checks some accessibility issues |

---

## 22. Best Memory Rules for Sprint 10

### Rule 1

> Accessibility means everyone should be able to use the page.

### Rule 2

> Start with semantic HTML.

### Rule 3

> Use real elements for real jobs.

Examples:

- Use `a` for links.
- Use `button` for buttons.
- Use `nav` for navigation.
- Use `main` for main content.

### Rule 4

> Do not make mouse-only pages.

### Rule 5

> If users cannot see or hear something, provide another way.

### Rule 6

> Automated tools help, but manual testing is still needed.

### Rule 7

> Valid and meaningful HTML is the first layer of accessibility.

---

## 23. Final Sprint 10 Summary

Sprint 10 teaches you the foundation of accessibility.

The main idea is:

> A web page should be usable by different people, different devices, and different assistive technologies.

You learned that accessibility supports users with:

- Visual difficulties
- Auditory difficulties
- Motor difficulties
- Cognitive difficulties

You learned WCAG and POUR:

- Perceivable
- Operable
- Understandable
- Robust

You learned that assistive technologies include:

- Screen readers
- Braille output
- Large print keyboards
- Alternative pointing devices
- Screen magnifiers
- Voice recognition software

You learned that automated tools like Lighthouse, WAVE, and IBM Equal Accessibility Checker are useful, but they cannot replace manual testing.

Most importantly, you learned this core rule:

> Accessibility starts with clear, valid, semantic HTML.

Before moving to Sprint 11, always ask:

> Can users see it, hear it, reach it, understand it, and can assistive tools read it correctly?